RallyPoint

Milestone 1 — Written Reflection

Author: Phoebe Wang
Course: ACIS 498, Northwestern University
Milestone: Milestone 1 — Front-End Component
Date: June 22, 2026
Repository: github.com/phoebewong214/rallypoint
Live demo: app.tryrallypoint.com

This written reflection supports my Milestone 1 submission for RallyPoint, an Information Systems Capstone project that helps recreational tennis and pickleball players find compatible local partners. Because Milestone 1 is the front-end component, I frame the project around the user workflow, the React front-end architecture, the interface decisions, and the business value the interface makes visible. The backend, database, authentication, and matching API already exist as supporting infrastructure, but the emphasis here is how the front end turns those capabilities into a coherent information system: a user can sign in, search for partners, understand why a match is recommended, save players and courts, request a game, coordinate play at a court, and navigate their schedule. Where I make a market or technology claim I cite a numbered source at the end; where a number is an assumption rather than a measurement, I say so.

1. Self-Evaluation and Reflection

RallyPoint has pushed me to think less like someone building isolated screens and more like someone designing an end-to-end system. The front end is not just a visual layer; it coordinates identity, server state, loading states, failure states, persistent user preferences, filtering, and business workflow. That is the part I am most proud of in Milestone 1: the interface is already organized around the real task flow of a recreational player, not around a collection of unrelated pages.

Programming: 4/5. I built the React 19 + TypeScript front end with a shared API client, AuthContext, TanStack Query hooks, reusable components, route protection, lazy loading for the map-heavy Courts page, and a design-token-based CSS system. The most important architectural improvement was making the API base URL environment-driven. In development the app can talk to Flask at localhost; in production Vercel must inject VITE_API_URL pointing to the deployed Render API, and if that variable is missing in production the app now fails loudly instead of silently trying to call the user's own localhost. That is a small detail, but it is exactly the kind of environment separation a real deployed system needs. I also fixed TypeScript issues around SVG icon props and CSS imports, then added type checking to CI so those errors do not slip into future deployments.

User interface design: 4/5. The main user journey is usable and visually coherent: Find Partners shows ranked cards with a match tier, a percentage score, and reason chips that explain each match; filters update live with a debounced applied state; players set their weekly availability on a real preferred-times grid; Sessions organizes games into upcoming, requests, and past; Courts uses real Chicago court data

with map and distance context and now supports coordinating play through open games and check-ins; Profile includes saved players as a real persistent preference rather than only local browser state. I also implemented dark mode, skeleton loading, empty states, error states, and a mobile navigation drawer. I do not rate this 5/5 because the CSS is still concentrated in one large shared file. It is disciplined with tokens and naming rules, but the next professional step is to split tokens, shared components, and page-specific styles before the interface grows further.

Database systems: 3/5. Milestone 1 is front-end focused, so I am not claiming database work as the centerpiece. Still, the front end is designed around real persisted entities: the data model spans about twelve tables - users, sport profiles, courts, favorites, saved players, appointments and their participants, check-ins, game invites, sessions, availability slots, and AI match logs. From the UI side I learned how much a page's usefulness depends on whether the underlying model supports the user's action. Moving saved players from localStorage to a server-backed model, for example, made the Profile page more credible because a saved partner becomes part of the user's account, not just one browser's memory. My next growth area is backend query performance and migration maturity, which belongs more naturally to later milestones.

Business analysis: 4/5. RallyPoint addresses a real coordination problem: recreational players often have intent to play but cannot easily find someone with compatible skill, schedule, and court location. The front end is designed to reduce that friction visibly - the user does not just get a list, they get ranked partners, a fit percentage, a reason, court context, and a direct request flow. I have also started thinking in stakeholder and ROI terms (detailed in the Value Creation section): players get easier matches, public courts and parks departments could see better utilization, and the platform could eventually monetize premium discovery or local facility partnerships. The numbers in that section are illustrative assumptions, not measured traction, so the next step is validation through user testing and usage metrics.

Project management: 4/5. As a solo developer I had to sequence work by value rather than by what was easiest to build. I prioritized the core front-end loop first - authenticate, search, understand, save, request, review - then layered on court coordination. I also maintained documentation, deployment notes, and CI checks. The live deployment on Vercel + Render matters because it proves the app is not only a local prototype, and I added the operational pieces a real product needs - an admin dashboard and an in-app support widget - so the system can be run, not just demoed. My biggest project-management lesson is that Milestone 1 should not overclaim backend or AI maturity; it should clearly show a polished front-end information system and name the remaining backend and data work honestly.

Quick Programming Test score: 3/5.

Overall I would evaluate my Milestone 1 as strong for a first front-end milestone because it demonstrates a real user journey, an environment-aware deployed architecture, and several production-minded interface patterns. My main improvement areas are accessibility testing, component and style modularization, and deeper mobile QA on the filtering and map interactions.

2. Business Problem Identification

RallyPoint addresses a coordination problem in recreational sports. Tennis and pickleball participation have grown quickly, but finding the right partner is still fragmented. A player usually needs three things to line up before a game actually happens: comparable skill, compatible availability, and a realistic court location. Existing informal channels - group chats, Facebook groups, club bulletin boards, word of mouth - are not built to match on those dimensions. They broadcast intent, but they do not structure it.

The problem is especially clear for newer or casual players. A beginner or intermediate player may want to play regularly but not know who is at the same level. A strong player may want a competitive match but waste time on mismatched replies. Working adults may be free only at specific times, and city players may be in the same metro area while still too far apart to make a quick game realistic. The result is not a lack of interest; it is a failure of coordination.

This is an Information Systems problem because the issue is not only the absence of software - it is the absence of a system that captures the right data, presents it usefully, and supports a decision. RallyPoint's front end is designed around that decision: "Who should I play with next, and why is that person a good fit?" The answer combines profile data, sport rating, court proximity, availability summary, saved preferences, and session state. The interface translates raw data into a user's next action. Comparable platforms such as Playo show that the matching-plus-proximity model works broadly [4], but none is built around a single city's public-court network the way RallyPoint is.

The cold-start problem is also central. A partner-matching system needs local density; one user alone has little value. So RallyPoint's wedge is not "everyone everywhere" but city-by-city and court-by-court density. The current product focuses on Chicago public courts and treats courts as local hubs. This is a more defensible launch strategy than spreading thinly across a large geographic market, because public courts are fixed locations around which recurring players naturally cluster.

3. Demonstration of Solution

The front-end solution demonstrated in Milestone 1 is the core user workflow:

1. A user signs in through the live React app.

2. The app validates the session through AuthContext and the API client.

3. The user opens Find Partners and sees a ranked partner list.

4. Each card shows sport, rating, distance, a match tier and a percentage score, reason chips (for example same level, shared time slots, or similar playing style), and a one-line summary.

5. The user filters by sport, skill range, time preference, and home court.

6. The user can sort by best match, distance, or skill.

7. The user can bookmark a player, with the saved state persisted through the server.

8. The user can request a game through a two-phase invite - propose a time and pick a court, and the other player confirms or counters - so a request is never faked into a confirmed game.

9. The user can open a court's detail page to see who is here now, check in, and join or create an open game (with a waitlist when it is full).

10. The user can review upcoming games, incoming requests, and past activity in Sessions, and browse real public courts with distance and map context in Courts.

The important design choice is that this is not just a landing page. The first meaningful screen after login is a task surface: it gives the user enough information to decide, then the action that naturally follows. That makes the front end behave like a working information system rather than a marketing mockup.

Several front-end states are also part of the demonstration. Loading states use skeleton cards, so the page does not jump from blank to content. A real empty response shows "No partners match these filters yet," which is different from an API failure. If the backend is unreachable, the page shows a retry state in production. Sample-partner fallback is limited to development unless VITE_DEMO_FALLBACK is explicitly enabled, because a deployed information system should not silently show fabricated users as if they were real.

4. Front-End Code Walkthrough Summary

The front-end architecture has a clear line.

The API client is the foundation. All requests go through frontend/src/api/client.ts. It reads VITE_API_URL from the environment, sends credentials with every request so the browser includes the httpOnly session cookie, adds X-CSRF-Token on unsafe methods, parses JSON consistently, throws ApiError on failures, and broadcasts auth:expired on 401 responses. The production and development split is explicit: localhost is only a Vite development fallback, while production requires Vercel to inject the deployed API URL.

AuthContext is the identity layer. It stores only a non-secret user object locally for quick first paint; it never stores the JWT in JavaScript. On mount it calls /auth/me to validate the httpOnly cookie and refresh user state, and it listens for global auth expiration and cross-tab logout. ProtectedRoute then uses that centralized state to gate authenticated pages and the email-verification flow.

TanStack Query manages server state. Hooks such as usePlayers, useSessions, useCourts, useCourtDetail, and useSavedPlayers keep data fetching out of the page components. Query keys include filters where appropriate, so the same UI can cache and refetch based on the user's choices. Mutations such as the saved-player and favorite toggles use optimistic updates, so the UI feels immediate while still rolling back on failure.

FindPartnerPage is the core product surface. It separates draft filters from applied filters with a 300ms debounce, which prevents the NTRP slider from firing a backend request on every pixel of movement. It projects backend player data into card props, preserves true empty responses, distinguishes an unavailable API from a no-match result, and renders the match explanation exactly as the backend returns it - the front end never recomputes the score.

The shared design layer lives in rally-shared.tsx and rally-shared.css: shared icons, TopNav, Avatar, design tokens, dark-mode variables, skeletons, toasts, and modals. This gives the project a consistent

visual language. The limitation is that the CSS file is large, so future work should move toward more modular component styles.

5. Discussion of Issues and Resolutions

Issue 1: Production API configuration could silently fall back to localhost. Early versions used VITE_API_URL if present and otherwise defaulted to http://localhost:5050/api. That was convenient locally but created a production risk: if Vercel was missing the variable, the deployed app would ask the user's browser to call its own localhost. I fixed this with resolveApiBase(): the localhost fallback only exists when import.meta.env.DEV is true, and in production a missing VITE_API_URL throws an explicit error. I also updated README, DEPLOY.md, and the video script so the deployment story is clear - Vercel injects the front-end API URL, Render hosts the API, and localhost is only for local development.

Issue 2: Demo data could blur the line between sample content and real users. The Find Partners page originally had a sample-data fallback for when the backend was down or unseeded. That is useful in development but dangerous in production, because a real user could mistake fabricated players for real ones. I changed the logic so sample players only appear in Vite dev mode or when VITE_DEMO_FALLBACK=true is explicitly set. In production an API failure now shows a clear unavailable state with a Try Again button, while a true empty response still shows a no-match message. This makes the UI more honest.

Issue 3: TypeScript was not enforcing the front-end contract strongly enough. A type-check failure came from a conflict between SVGProps and the custom Icon stroke prop, and CSS side-effect imports needed declarations. I fixed IconProps by omitting the inherited SVG stroke before adding the app-specific numeric stroke prop, added vite-env.d.ts, and added npm run typecheck to CI. This is small engineering hygiene, but a front end with typed API contracts should not rely only on a Vite production build catching errors.

Issue 4: Saved players started as a local-only interaction. Bookmarking a player is a meaningful preference, so keeping it only in localStorage would make it feel temporary and device-specific. I moved the flow to a server-backed saved-players API with useSavedPlayers and useToggleSavedPlayer hooks. The Find Partners card now reads a backend saved flag, and Profile can show saved players from the account. The mutation uses optimistic updates so the UI feels instant while the server remains the source of truth.

Issue 5: The interface needed to communicate AI honestly - real where it helps, never a fake label. An earlier version showed an "AI Match" badge over what was really a heuristic, so I removed the badge in favor of a transparent tier plus reason chips that show exactly why two players match. I then added genuine AI where it actually improves the result: a semantic "similar playing style" signal computed from OpenAI embeddings of each player's bio (cosine similarity), which contributes to the score but degrades gracefully to nothing when a bio or API key is missing. The optional LLM still only rewrites the reason text, never the ranking. The result is AI that is real, auditable, and explainable - not a black box and not a marketing label.

6. Value Creation

RallyPoint creates value by reducing coordination friction, and I tried to trace that value all the way to a concrete outcome rather than leaving it as adjectives. For broad context, the McKinsey Global Institute estimates generative AI could add $2.6 to $4.4 trillion in value annually across the use cases it analyzed [3]; that is the macro backdrop, not RallyPoint's own ROI, which I work out specifically below.

For players, the product saves time and improves confidence: a person is not browsing random profiles, but seeing a ranked list with reasons based on skill, distance, and availability. That should increase the chance a first session turns into recurring play. Illustratively (an assumption to validate, not a measurement), if 40% of well-matched first sessions convert to recurring play versus roughly 15% for unmatched attempts, that activation lift is the player-side value.

For public courts and local facilities, the central measurable unit is recovered idle court-hours. RallyPoint does not reserve courts today, so I do not claim direct booking control; the effect is indirect - it surfaces and steers demand toward real locations. One worked value chain, end to end with stated assumptions: take a starter base of 300 active local players clustered around a few public-court hubs; assume each plays roughly one RallyPoint-matched session per week, and that about 30% of those are incremental games that would not have happened without a match found. That is 300 x 1 x 0.30 = 90 incremental matched sessions per week; at an average of 1.5 court-hours per session, roughly 135 recovered court-hours per week routed onto otherwise-idle courts. The biggest sensitivity is the incremental-play rate, and every number here is an assumption to validate.

For the platform, the value is a location-anchored network. Saved players, court favorites, sessions, check-ins, and match interactions can become a data asset over time. With enough local density, monetization could come from a freemium model - illustratively, 5% conversion x $4 per month x a retained base of 1,000 users is about $200 per month - plus premium matching filters, advanced scheduling, or partnerships with organizations that manage recreational sports spaces.

The value also comes from trust. A match score alone is not enough; users need to understand why someone is recommended. The reason text lowers uncertainty, especially for a new user deciding whether to request a game. That is why the hybrid AI design matters: a transparent, explainable score - now including a genuine semantic "playing style" signal - protects correctness and cost, while the reason chips and optional explanation layer make the recommendation legible, the lever that turns a recommendation into a booked session.

7. Technology Trends and Industry Context

The first and most central trend is AI-assisted recommendation and explanation, and because the matching engine is the product's defining technology choice, this milestone should justify it rather than just name it. I evaluated four approaches against the criteria that actually matter here - value, cost, latency, explainability, maintainability, and data readiness:

- Pure heuristic: solid, auditable ranking at near-zero cost and instant latency, but the reasons read as terse and mechanical.

- Pure LLM ranker: rich explanations, but a token charge and a network round-trip on every match, plus the risk of a hallucinated justification - unacceptable as the ranker because correctness and auditability are not guaranteed.
- Hybrid (chosen): a transparent, explainable score built from real signals - skill closeness (the dominant signal), court proximity by great-circle distance, weekly preferred-times overlap from the actual availability grid, and a shared home court - plus a genuine AI signal: a "similar playing style" score from the cosine similarity of OpenAI embeddings of each player's bio. Every signal also emits a human-readable reason chip, so the score is auditable and the UI can show why two players match. The embedding signal degrades gracefully to zero when a bio or API key is missing, so the AI only ever adds information. A separate, optional gpt-4o-mini call (roughly $0.0005 to $0.002 and 300-800 ms, grounded by a "never invent facts" prompt) can rewrite the reason text, but it never affects the score. So the hybrid earns genuine AI value - semantic style matching plus optional natural-language reasons - without handing the ranking to a black box.
- Trained ML ranking model: potentially the best ranking, but it needs a labeled history of accepted and declined matches that does not exist yet. That is the right future option once enough real interaction data accumulates - which is exactly what the ai_match_logs table is designed to capture, though that logging is not yet wired onto the served path.

The conclusion is that the hybrid is the maximum-value approach today: real AI where it improves the result, full transparency everywhere, and a designed-but-not-yet-running data path to the learned ranker - an honest "explainable now, learning later" sequence.

The second trend is geolocation and proximity discovery. Ride-sharing, delivery, and local-event apps have trained users to expect location-aware experiences. A partner is only useful if the court is realistically close, so distance and court context are not decorative - they are part of the core matching decision.

The third trend is mobile-first workflow design. Recreational coordination happens quickly - a player checks availability after work, saves a partner, or answers a request from a phone. The app already includes responsive layout and a mobile navigation drawer; the next step is more thorough mobile QA, especially around filtering, map interaction, and scheduling.

The fourth trend is cloud deployment and environment separation. RallyPoint runs on Vercel for the React front end, Render for the Flask API and Postgres database, and Resend for transactional email. That is an appropriate MVP cloud architecture because it keeps operational complexity low while proving the app runs outside localhost, and the front-end requirement for VITE_API_URL in production is part of this trend: deployed software needs explicit environment configuration.

From an industry perspective, the participation data supports the problem. SFIA reported that 247.1 million Americans were active in at least one activity in 2024, with pickleball reaching 19.8 million participants and strong year-over-year growth [1]; the USTA reported record tennis participation of 25.7 million players in 2024 [2]. These do not prove RallyPoint will succeed, but they show the trend behind the opportunity: more players create more coordination demand.

8. Risks and Mitigations

Naming risks - and what already mitigates them - is part of treating this like a real system rather than a class project.

- Cold-start / two-sided liquidity: the product has little value until enough compatible players exist nearby. Mitigation: the public-court-hub density wedge - saturate one city (Chicago) court by court before expanding, rather than spreading thin.
- LLM cost, latency, and hallucination: mitigated by the deterministic backbone that carries all ranking, the grounded "never invent facts" prompt, the token cap, and the API-key-gated graceful fallback to the heuristic reason. The persist-and-reuse cache in ai_match_logs would further bound token spend, but it is designed, not yet wired, so I count it as a planned mitigation.
- Location privacy: mitigated by using coarse, approximate distance rather than exposing exact coordinates.
- Security: addressed in shipped code - the JWT lives in an httpOnly cookie (unreachable from JavaScript) with a double-submit CSRF token on unsafe methods, plus Flask-Limiter rate limiting on auth and Pydantic validation on request bodies.
- Honesty / overclaiming: a real risk for a demo graded by someone who can read the code. Mitigation: I removed the fake "AI Match" badge, and the reflection and demo are explicit that the score is a transparent heuristic plus a real (but gracefully optional) embedding signal, that the live LLM only rewrites reason text and is off in the demo, and that match logging is not yet live.

9. Capstone Project Ideas and Selection

Idea 1 - RallyPoint: AI-powered partner matching for recreational tennis and pickleball. This is the selected project. It combines a clear user workflow, front-end interaction design, a real data model, location context, and a hybrid recommendation system. It fits an Information Systems Capstone because it connects people, data, processes, and technology around a real coordination problem, and Section 7 shows why the hybrid matcher is the maximum-value technology approach.

Idea 2 - Public-court utilization dashboard. This would focus on courts rather than players: tracking activity, surfacing peak times, and helping parks departments understand usage. It has strong institutional value but would require operational data that is harder for a solo student project to obtain.

Idea 3 - Community sports league manager. This would manage ladders, standings, rosters, and pickup events for recreational groups, comparable to activity-first communities like Meetup. It is useful and feasible but less differentiated, because many event- and league-management tools already exist.

I selected RallyPoint because it has the strongest combination of user pain, technical differentiation, and capstone scope. It is small enough to build solo but rich enough to demonstrate front-end architecture, API integration, data modeling, deployment, AI-assisted recommendation, and business reasoning. Most importantly, the front end already makes the business process visible: a user can move from intent to partner discovery to game request inside one coherent system. The forward moat is the data it accrues - saved players, sessions, check-ins, and (once logging is wired) match outcomes - an asset the booking-only and event-only alternatives do not accumulate.

10. Next Steps

After Milestone 1, the next steps are:

1. Wire match logging and accept/decline outcomes into ai_match_logs on the served path, then move toward a learning-to-rank loop once there is enough real interaction data.

2. Make sure every player's bio embedding is populated (a background/batch job) so the semantic "similar playing style" signal is always available, not only when a bio is embedded at write time.

3. Add accessibility testing for keyboard navigation, focus order, color contrast, and screen-reader labels.

4. Continue modularizing the large shared CSS file into design tokens, shared components, and page-level styles.

5. Expand real user testing and deeper mobile QA around the Find Partners filter flow and the invite, appointment, and court-picker modals.

6. Tune backend query performance and indexes as real-user data grows.

Two earlier next steps are already done: the availability signal now uses the structured weekly preferred-times grid instead of keyword overlap, and production schema management has moved to Flask-Migrate (Alembic) migrations.

Milestone 1 proves the front-end workflow and the user-facing value. The next milestones should deepen the backend data pipeline and improve the system's ability to learn from real usage.

References

[1] Sports & Fitness Industry Association. "SFIA's Topline Participation Report Shows 247.1 Million Americans Were Active in 2024." https://sfia.org/resources/sfias-topline-participation-report-shows-247-1-million-americans-were-active-in-2024/

[2] USTA. "U.S. Tennis Participation Grows to Record 25.7 Million Players in 2024." https://www.usta.com/en/home/stay-current/national/u-s--tennis-participation-grows-to-record-25-7-million-players-i.html

[3] McKinsey Global Institute. "The economic potential of generative AI: The next productivity frontier." https://www.mckinsey.com/capabilities/mckinsey-digital/our-insights/the-economic-potential-of-generative-ai-the-next-productivity-frontier

[4] Playo. Official site, cited as an example of a sports discovery and play-coordination platform. https://playo.co/